

Sovereign AI --- PS 26117

Improved Blueprint: Fully Aligned with Problem Statement

Purpose: Close the gaps between the original blueprint and the full PS 26117 problem statement. **Changes:** Added PPT generation, correspondence/email RAG, network monitor UI, expanded task taxonomy, explicit code artifact delivery, and elevated multimodal demo to equal priority.

1. Executive Summary

PS 26117 asks for a sovereign, on-premise AI workbench that behaves more like a private enterprise agent than a simple chatbot.

The system must:

- Run on the organization's own infrastructure.
 - Support multiple open-weight models.
 - Route tasks to suitable models automatically.
 - Execute local tools with iteration.
 - Understand text, scanned documents, handwritten notes, engineering drawings, and photographs.
 - Ground answers in a local knowledge base containing SOPs, manuals, **and correspondence**.
 - Generate real deliverables: approval notes, spreadsheets, calculations, **presentations (PPT)**, and code.
 - Keep confidential information inside the deployment boundary.
-

2. Final Technology Stack

Layer	Technology	Notes
Frontend	Lovable + React + TypeScript + Tailwind + shadcn/ui	
Backend API	Python + FastAPI + Pydantic	
Agent orchestration	LangGraph	Stateful workflows, planning, tool calls, retries
Model serving	vLLM	HTTP/OpenAI-compatible APIs
General LLM	Small local Qwen instruct-class model	General reasoning, summarization, tool decisions
Coding LLM	Qwen2.5-Coder-3B-Instruct	Code generation, debugging, data-analysis scripts
Vision model	Small local Qwen-VL/VLM-class model	Images, scanned pages, engineering drawings
Embeddings	BGE/E5 family, local	Dense semantic retrieval
Sparse retrieval	BM25	Exact identifiers, SOP codes, part numbers
Fusion	Reciprocal Rank Fusion (RRF)	Combines dense and sparse rankings
Reranking	Local BGE-style reranker	Improves top-k relevance
Vector DB	Qdrant	Hybrid retrieval with metadata
Document parser	PyMuPDF	Digital PDF text and page metadata
OCR	PaddleOCR	Local OCR for scanned PDFs/images
Office generation	python-docx + openpyxl + python-pptx	Word, Excel, PowerPoint deliverables
Sandbox	Piston + Docker	Isolated code execution
Relational DB	PostgreSQL	Users, tasks, executions, audit records
Storage	Local filesystem / optional MinIO	Confidential source files and artifacts

Layer	Technology	Notes
Containers	Docker + Docker Compose	Repeatable local deployment

Change: Added `python-pptx` to office generation layer.

3. Architecture Principles

3.1 Local-first

Every sensitive operation has a local implementation.

3.2 Model abstraction

Application logic never hard-codes a single model. New models register via the Model Registry.

3.3 Tool isolation

LLM requests tools, but tools enforce their own validation and permissions.

3.4 Evidence-first RAG

Retrieved chunks carry document/page/section metadata for citations.

3.5 Observable execution

Every agent step records: task, model, tool, duration, status, artifact, metadata.

3.6 Controlled autonomy

Agent plans and iterates through an allow-listed tool registry only.

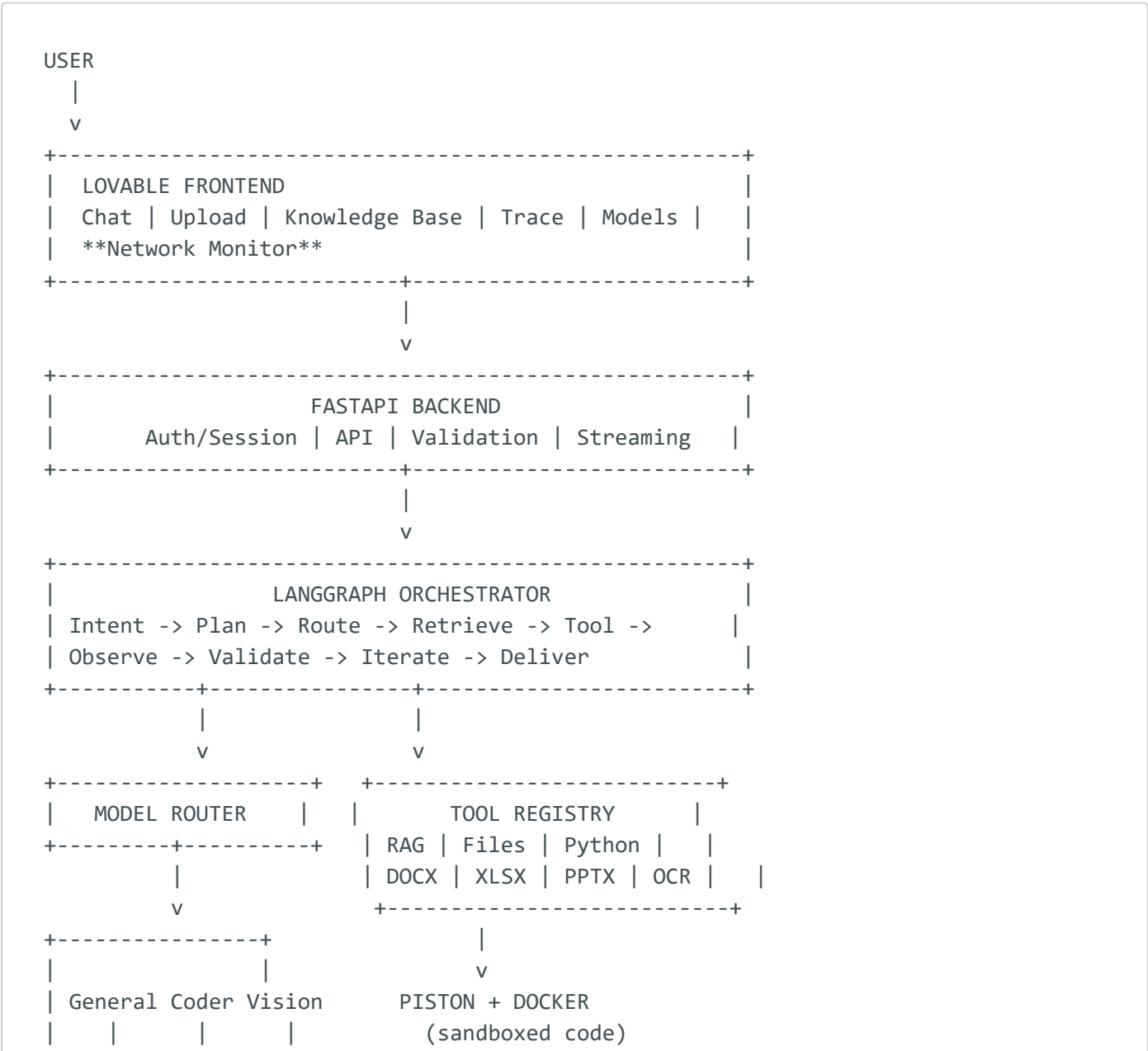
3.7 Hardware-aware serving

Single mid-range GPU (6 GB RTX 4050) constraint: use quantized models, sequential loading.

3.8 Sovereignty proof on demand

Change: The system must expose a **real-time network monitor** visible in the UI showing outbound connections attempted and blocked. Logs alone are insufficient for proof — the UI must make sovereignty visible during demos.

4. High-Level System Architecture



```
+-----+-----+
|
|
v
LOCAL MODEL SERVING (vLLM)
```

RAG PATH:

Document

- > Parse/OCR
- > Chunk
- > Dense Embedding
- > Sparse/BM25
- > Qdrant
- > RRF
- > Reranker
- > Context

CORRESPONDENCE PATH (NEW):

Email/Letter

- > Extract headers + body
- > Chunk
- > Dense + BM25
- > Qdrant
- > RAG fusion
- > Context

DATA PATH:

Local Files

- <-> PostgreSQL metadata
- <-> Qdrant vectors

5. Backend Architecture --- Python + FastAPI

5.1 API Surface

Endpoint	Purpose
POST /api/chat	Conversational request, stream final/trace events
POST /api/agent/run	Complete agent workflow
POST /api/documents/upload	Upload and register a document
POST /api/documents/ingest	Parse/OCR/chunk/embed/index a document

Endpoint	Purpose
GET /api/documents	List knowledge-base documents
POST /api/rag/search	Debug/test hybrid retrieval
GET /api/models	Return available local models and capabilities
POST /api/models/route	Return selected model for a task
POST /api/sandbox/execute	Submit code to Piston through controlled adapter
GET /api/executions/{id}	Return execution trace
GET /api/artifacts/{id}	Download generated deliverable
GET /api/system/status	Show local services/model/vector DB health
GET /api/network/monitor	Return real-time outbound connection attempts and blocked calls

5.2 Backend Modules

```

backend/
├── app/
│   ├── main.py
│   └── api/
│       ├── chat.py
│       ├── documents.py
│       ├── rag.py
│       ├── models.py
│       ├── sandbox.py
│       ├── executions.py
│       └── network.py           # NEW: network monitor endpoint
│   ├── agents/
│       ├── graph.py
│       ├── state.py
│       ├── planner.py
│       ├── router.py
│       └── policies.py
│   ├── models/
│       ├── registry.py
│       ├── client.py
│       └── capabilities.py
│   └── rag/
│       ├── ingest.py
│       ├── parser.py
│       ├── ocr.py
│       └── chunker.py

```

```
├── dense.py
├── sparse.py
├── fusion.py
├── reranker.py
├── citations.py
├── correspondence.py    # NEW: email/letter processing
├── tools/
│   ├── rag_tool.py
│   ├── file_tool.py
│   ├── python_tool.py
│   ├── docx_tool.py
│   ├── spreadsheet_tool.py
│   └── pptx_tool.py      # NEW: PowerPoint generation
├── security/
│   ├── sandbox_policy.py
│   ├── network_policy.py
│   ├── network_monitor.py  # NEW: real-time connection tracker
│   └── audit.py
├── storage/
│   ├── postgres.py
│   ├── qdrant.py
│   └── files.py
├── schemas/
│   ├── api.py
│   ├── agent.py
│   └── documents.py
├── tests/
├── Dockerfile
└── requirements.txt
```

6. Expanded Task Taxonomy

Problem Statement implies diverse task types. Original blueprint had 7 classes; this expands to 12.

Task Class	Description	Primary Model	Output
GENERAL_QA	Conversational Q&A	General LLM	Text response
RAG_QA	Question grounded in internal docs	General LLM + RAG	Text + sources
DOCUMENT_ANALYSIS	Inspection reports, memos, letters	General LLM + OCR	Structured summary
MULTIMODAL_ANALYSIS	Engineering drawings, photos,	VLM	Visual description +

Task Class	Description	Primary Model	Output
	scanned docs		analysis
CODING	Code generation, debugging	Coder LLM	Code artifact + sandbox result
DATA_ANALYSIS	CSV analysis, calculations	Coder LLM + sandbox	XLSX + verified result
DOCUMENT_GENERATION	Approval notes, formal letters	General LLM + tools	DOCX artifact
PRESENTATION_GENERATION	Board presentations, reviews	General LLM + tools	PPTX artifact
SPREADSHEET_WORK	Tables, calculations	General LLM + openpyxl	XLSX artifact
CORRESPONDENCE_SEARCH	Email/letter retrieval	General LLM + RAG	Evidence + context
ITERATIVE_REVIEW	Multi-step review with corrections	Any (iterative)	Full trace + artifact
CALCULATION	Engineering calculations with steps	Coder LLM + sandbox	Steps + verified result

7. Model Registry and Router

7.1 Registry entries must declare

- Capabilities
- Resource requirements (VRAM, context length)
- Endpoint
- Status
- Supported task classes

7.2 Routing Logic

The router uses deterministic rules (explainable) enhanced by task classification:

```
IF image/drawing is present AND task involves visual understanding
    -> select VLM

IF task type is CODING or DATA_ANALYSIS or CALCULATION
    -> select Coder LLM

IF task type is RAG_QA or CORRESPONDENCE_SEARCH
    -> enable RAG + General LLM

IF task type is DOCUMENT_GENERATION or PRESENTATION_GENERATION
    -> select General LLM + appropriate output tool

IF task type is MULTIMODAL_ANALYSIS
    -> select VLM

DEFAULT
    -> General LLM
```

Model registry and router allow adding new models without redesign — register new model with capabilities, routing rules pick it up automatically.

8. Hybrid RAG --- With Correspondence Support

8.1 Document RAG (unchanged)

Parse → Clean → Structure-aware Chunk → Metadata → Dense Embedding → Sparse/BM25 → Qdrant → RRF → Reranker → Context

8.2 Correspondence RAG (NEW)

Problem Statement mentions "past correspondence" but original blueprint only covers PDF/DOCX/XLSX.

```
Email/Letter file (.eml, .msg, .txt)
↓
Header extraction (from, to, date, subject, thread_id)
↓
Body text extraction
↓
Structure-aware chunking (preserve thread context)
↓
Dense embedding + BM25
↓
Qdrant (with correspondence-specific metadata: sender, recipient, date_range)
↓
Same RRF + Reranker pipeline
↓
Context with correspondence provenance
```

Supported input formats for correspondence:

- **.eml** (standard email format)
- **.msg** (Outlook)
- Plain text letters with header lines

8.3 Unified retrieval

Both document RAG and correspondence RAG feed into the same fusion pipeline. A query about "vendor negotiations with Acme Corp" can retrieve both SOPs mentioning Acme and relevant email threads.

9. Multimodal Document Intelligence

9.1 Processing rules

Input	Pipeline
Digital PDF	PDF → PyMuPDF → text
Scanned PDF	PDF → Page Render → PaddleOCR → text
Image (drawing/photo)	Image → Local VLM → Visual description + analysis

Input	Pipeline
Handwritten notes	Image → PaddleOCR → text (fallback to VLM if OCR fails)
<code>.eml/.msg/.txt</code> correspondence	Correspondence pipeline above

9.2 Normalized representation

```

FILE
+-- text PDF -----> PyMuPDF --> text
+-- scanned PDF -----> page render --> PaddleOCR --> text
+-- image/drawing ----> local VLM --> visual description
+-- email/letter -----> correspondence parser --> structured text
v
normalized document representation
+--> RAG indexing
+--> direct task analysis
+--> artifact generation

```

10. Coding Agent + Secure Sandbox

10.1 Workflow

```

USER: "Analyze this CSV and calculate average downtime by machine."

Task classifier
↓
CODING / DATA ANALYSIS
↓
Qwen2.5-Coder-3B-Instruct
↓
Generated Python
↓
Static / Policy Validation
↓
Piston API → Docker Sandbox (no network, timeout, CPU/RAM limits, isolated FS)
↓
stdout / stderr / exit code
↓
Agent validates result
|--> retry/fix if allowed

```

↓
Final result + code artifact (downloadable .py file)

10.2 Code as deliverable

Problem Statement says "working code" as output. The sandbox result is the verified execution, but the **source code itself must be saved as a downloadable artifact** (.py file linked to the execution record).

11. Real Deliverables

Workflow	Input	Output
Inspection approval	Scanned inspection report + SOP KB	approval_note.docx + evidence
Data analysis	CSV + natural-language request	Verified result + .py code artifact + optional XLSX
Engineering image	Drawing/photo + query	Visual explanation + execution trace
Board presentation	Topic + source docs	presentation.pptx
Spreadsheet work	Data + instructions	data.xlsx with calculations
Code deliverable	Task description	Verified .py artifact + execution output

All artifacts: stored locally, assigned artifact ID, linked to execution record, downloadable from UI.

12. Sovereignty, Security & Air-Gapped Runtime

12.1 Network monitor (NEW)

Problem Statement requires "visible network monitor." This must be a real-time UI component showing:

- All outbound connection attempts (destination host, port, timestamp)
- All connection outcomes (blocked/allowed)
- Cumulative count: **External AI API calls: 0**

Implementation:

- A background process intercepts all outbound connections from the Docker network
- A **GET /api/network/monitor** endpoint streams events via SSE
- Frontend renders a live network monitor panel

12.2 Audit log

Maintains record of:

- Model invocations
- Tool invocations
- Execution steps
- External network calls (should always be 0)
- Artifacts generated

12.3 Security boundary

```
NO CLOUD AI API
NO CLOUD VECTOR DB
NO CLOUD OCR
NO CLOUD FILE STORAGE
NO CLOUD EMBEDDINGS
NO CLOUD RERANKING
```

All inference, embeddings, retrieval, OCR, file storage, and tool execution remain inside the Docker Compose boundary.

13. Execution Trace & Observability

Every step emits an event:

```
{
  "type": "step",
  "step": "hybrid_rag",
  "status": "SUCCESS",
  "model": "local-general-llm",
  "tool": "qdrant_hybrid_search",
  "duration_ms": 1200,
  "external_network_calls": 0,
  "evidence": ["Maintenance_SOP.pdf p.17", "Vendor_Email_2024-03.eml"]
}
```

Frontend renders a compact trace:

```
✓ task classified: DOCUMENT_ANALYSIS
→ ✓ model selected: general-llm
→ ✓ OCR completed (PaddleOCR, local)
→ ✓ hybrid retrieval: 5 chunks found
→ ✓ SOP evidence retrieved (p.17, p.23)
→ ✓ analysis complete
→ ✓ DOCX generated
→ ✓ artifact stored: approval_note.docx
→ ✓ external calls: 0
```

14. Core Data Model

14.1 Entities

Entity	Key fields
User	id, role, created_at
Conversation	id, user_id, title, created_at
Message	id, conversation_id, role, content, created_at
Document	id, filename, mime_type, size, checksum, doc_type (pdf, correspondence, image), status, created_at

Entity	Key fields
DocumentChunk	id, document_id, page, section, text, metadata, qdrant_point_id
AgentExecution	id, conversation_id, task_type, status, started_at, completed_at
ExecutionStep	id, execution_id, step_index, action, model, tool, status, duration, metadata
Artifact	id, execution_id, filename, mime_type, path, checksum
Model	id, name, endpoint, capabilities, context_length, status
NetworkEvent	id, timestamp, destination_host, destination_port, action (blocked/allowed), execution_id

NEW: NetworkEvent entity for real-time network monitoring.

15. Four Golden Demos (Equal Priority)

Demo 1: Inspection Report → Approval Note

- 1. Upload scanned inspection report (PDF)
- 2. PaddleOCR extracts text
- 3. Hybrid RAG against maintenance SOP
- 4. General LLM analyzes findings vs SOP
- 5. python-docx generates approval note
- 6. Download .docx artifact

Trace visible: OCR → retrieval → evidence → analysis → artifact

Demo 2: Data Analysis → Verified Code

- 1. Upload CSV
- 2. Natural language: "Calculate average downtime by machine"
- 3. Task classified as DATA_ANALYSIS

- 4. Qwen2.5-Coder generates Python
- 5. Piston/Docker executes in sandbox
- 6. Result verified and returned
- 7. Downloadable `.py` code artifact

Trace visible: classification → model selection → code gen → sandbox → verified result

Demo 3: Multimodal (Engineering Drawing)

- 1. Upload engineering drawing (image or scanned PDF)
- 2. VLM processes image locally
- 3. Returns structured analysis (dimensions, components, compliance notes)
- 4. Optionally combine with SOP context via RAG

Trace visible: image input → VLM → visual analysis

Demo 4: Correspondence Search (NEW)

- 1. Upload email archive (.eml files)
- 2. Ingest via correspondence pipeline
- 3. Ask: "What was discussed about Acme Corp vendor negotiations?"
- 4. Hybrid RAG retrieves relevant emails + SOP references
- 5. General LLM synthesizes answer with sources

Trace visible: correspondence ingestion → retrieval → evidence → synthesis

16. Frontend Screens

Screen	Purpose
Dashboard	System health, model status, recent executions, network monitor , sovereignty indicator
AI Workbench	Chat, file attachments, task submission, streaming response

Screen	Purpose
Knowledge Base	Upload documents/correspondence, ingestion status, metadata
Execution Trace	Agent steps, model routing, tools, evidence, timing, external calls count
Model Registry	Available models, capabilities, health, resource notes
Artifacts	Generated DOCX/XLSX/PPTX/code files with downloads
Network Monitor	Live panel: outbound connection attempts, blocked status, cumulative count

17. Docker Compose Deployment

```
services:
  frontend:
    build: ./frontend

  backend:
    build: ./backend
    depends_on:
      - postgres
      - qdrant
      - piston

  postgres:
    image: postgres

  qdrant:
    image: qdrant/qdrant

  piston:
    # self-hosted Piston stack

  vllm-general:
    # local general model (quantized, ~3-4GB VRAM)

  vllm-coder:
    # local coding model (quantized, ~3GB VRAM)

  vllm-vision:
    # local VLM where hardware permits

  network-monitor:
    # NEW: lightweight service tracking outbound connections
    # Intercepts at Docker network level
```

Hardware note: RTX 4050 (6 GB) can run one model at a time. Use sequential loading. All models quantized (AWQ/GGUF).

18. Implementation Phases

Phase	Goal	Exit Condition
1	Repository + Docker + FastAPI + Lovable shell	UI talks to <code>/api/system/status</code>
2	Local model serving + model registry	Local LLM returns an answer
3	Document ingestion + OCR	Scanned PDF becomes structured text
4	Hybrid RAG	Question retrieves correct local evidence
5	LangGraph agent	Task plans and tool calls visible
6	Coding sandbox	Generated code executes safely, returns result
7	DOCX/XLSX/PPTX artifacts	Agent produces real deliverables
8	Vision workflow	Image/drawing query works locally
9	Correspondence RAG	Email/letter ingestion and retrieval
10	Network monitor UI	Real-time visible proof of no external calls
11	Sovereignty proof + polish	All 4 demos work repeatedly

19. What NOT to Build in v1

Do **not**:

- Build 15 autonomous agents
- Train an LLM from scratch
- Support every document format (prioritize: PDF, DOCX, XLSX, PPTX, CSV, image, EML)

- Make cloud services part of the sovereign runtime
- Implement enterprise RBAC/SSO unless core functionality is complete
- Execute generated code inside the backend process
- Add a second vector database
- Spend final days changing models

Instead:

- Use one orchestrator with specialized tools/workflows
- Use open-weight models (quantized for 6 GB GPU)
- Prioritize PDF, image, CSV/XLSX, PPTX, DOCX, EML
- Keep generated code behind the sandbox boundary
- Freeze the model set early

20. Final Architecture Checklist

- ☐ Lovable frontend connected to FastAPI
- ☐ FastAPI API + streaming execution events
- ☐ LangGraph agent state and workflow
- ☐ Model Registry + Model Router
- ☐ Local general LLM
- ☐ Qwen2.5-Coder-3B-Instruct
- ☐ Local VLM
- ☐ Local embeddings
- ☐ BM25 sparse retrieval
- ☐ Qdrant hybrid retrieval
- ☐ RRF fusion
- ☐ Local reranker
- ☐ PyMuPDF
- ☐ PaddleOCR
- ☐ Piston + Docker sandbox
- ☐ python-docx artifact generation
- ☐ openpyxl artifact generation
- ☐ **python-pptx artifact generation (NEW)**
- ☐ PostgreSQL metadata/audit
- ☐ Local filesystem artifacts
- ☐ Execution trace
- ☐ **Network monitor real-time UI (NEW)**

- ☐ **Correspondence ingestion and RAG (NEW)**
- ☐ **Code artifact download (.py file) (NEW)**
- ☐ **Four golden demos (all equal priority) (EXPANDED)**
- ☐ Sovereignty/network proof visible in UI

21. One-Sentence Architecture

Lovable provides the local enterprise workbench UI; FastAPI exposes the application API; LangGraph orchestrates controlled agent workflows; a model router selects local open-weight LLM/VLM endpoints served by vLLM; hybrid RAG combines dense embeddings and BM25 with RRF and reranking over Qdrant; correspondence RAG handles email and letters; Piston/Docker executes generated code safely; python-docx/openpyxl/python-pptx produce real deliverables; PostgreSQL stores metadata/audit records; local storage holds confidential files and artifacts; a real-time network monitor makes sovereignty visible; Docker Compose packages the stack; and network isolation proves the sovereign runtime boundary.

22. Alignment Summary

Problem Statement Requirement	Covered In
Self-hosted, air-gapped	Sections 12, 20
Not locked to one model	Sections 7, 8
Multiple open-weight models	Section 2
Auto-select right model per task	Sections 7.2, 8.2
Add models without redesign	Sections 7, 8.2
Agentic (plan, iterate, multi-step)	Sections 5, 6, 7.2
Local tools (file, code sandbox, spreadsheet, search)	Sections 5, 10, 11
Scanned PDFs, handwritten notes, drawings	Sections 9, 10
Real deliverables (Word/Excel/ PPT /approval notes/code)	Sections 11, 15, 16

Problem Statement Requirement	Covered In
Local knowledge base (SOPs, manuals, correspondence)	Sections 8.2, 15 (Demo 4)
Visible network monitor	Sections 12.1, 16, 18 Phase 10
Four demos equally demonstrated	Section 15

All PS 26117 requirements are now covered with explicit implementations.